

Projet Data Science Finance

Projet : Système de monitoring de risque de portefeuille (VaR + détection de régime de volatilité)

Cadrage métier (à lire comme si tu étais sur un desk risk)

Tu es analyste risk sur un desk qui gère un portefeuille multi-actifs. Ta mission : construire un système qui, chaque jour, répond à trois questions pour le portfolio manager :

1. **Combien peut-on perdre demain, avec quelle confiance ?** (VaR)
2. **Notre méthode de calcul du risque est-elle fiable ?** (backtesting)
3. **Sommes-nous dans un régime de marché calme ou stressé ?** (classification ML, utile pour ajuster dynamiquement les limites de risque)

C'est un vrai problème de desk : la VaR seule ne suffit jamais — un risk manager sérieux la challenge (backtest) et cherche des signaux avancés (régime de volatilité) pour anticiper plutôt que subir.

Portefeuille de travail

Utilise ces 5 actifs (mêmes classes que dans notre matrice de corrélation) :

Actif	Ticker Yahoo Finance	Rôle
Actions US	SPY	Actions large cap
Obligations	TLT	Obligations d'État long terme
Or	GLD	Valeur refuge
Pétrole	USO	Matière première
Actions tech	QQQ	Actions plus volatiles

Portefeuille équilibré (20% chacun) pour rester simple — tu pourras complexifier plus tard.

Données

- Source : `yfinance` (gratuit, pas de clé API nécessaire).
- Historique : 5 ans de cours de clôture ajustés, quotidien.
- Aucune excuse pour ne pas avoir de vraies données : c'est la même API que tu utiliserais en stage.

```
import yfinance as yf
tickers = ["SPY", "TLT", "GLD", "USO", "QQQ"]
data = yf.download(tickers, period="5y", auto_adjust=True) ["Close"]
```

Pipeline à construire (dans l'ordre)

Étape 1 — Ingestion et nettoyage des données

- Télécharger les données, gérer les valeurs manquantes (jours fériés différents entre marchés, actifs introduits plus tard...).
- Calculer les rendements journaliers (`pct_change()`).
- Sauvegarder un fichier de données propre (`data/processed/returns.parquet`) pour ne pas re-télécharger à chaque run.
- **Test à écrire** : vérifier qu'il n'y a aucun NaN résiduel après nettoyage, que les dates sont bien ordonnées.

Étape 2 — Analyse exploratoire (EDA)

- Matrice de corrélation entre les 5 actifs (comme celle qu'on a vue) — est-ce cohérent avec l'intuition économique ?
- Courbe de densité des rendements du portefeuille — compare-la visuellement à une loi normale de même moyenne/écart-type (utilise `scipy.stats.norm` pour superposer la courbe théorique).
- Calcule skewness et kurtosis du portefeuille. Question à répondre dans ton rapport : **les rendements ont-ils des queues plus épaisses que la normale ?** (Réponds avec des chiffres, pas une impression.)

Étape 3 — Calcul de la VaR (3 méthodes, obligatoire de les comparer)

Pour un horizon 1 jour, confiance 95% et 99% :

1. **VaR paramétrique** (suppose une loi normale) : $VaR = -(\mu + z * \sigma)$
2. **VaR historique** : le quantile empirique des rendements passés (pas d'hypothèse de distribution).
3. **VaR Monte Carlo** : simule 10 000 scénarios à partir de la moyenne/covariance du portefeuille, prends le quantile de la distribution simulée.

Livrable attendu : un tableau comparant les 3 méthodes aux 2 niveaux de confiance, plus une phrase expliquant pourquoi elles divergent (ça doit faire écho à notre discussion sur les queues épaisses).

Étape 4 — Backtesting de la VaR (le vrai réflexe pro)

Sans ça, ton calcul de VaR n'est qu'une hypothèse non vérifiée.

- Compte le nombre de fois où la perte réelle a dépassé la VaR prédite ("exceptions"), sur toute la période historique, pour chaque méthode.
- Avec une VaR à 95%, tu attends ~5% de jours en exception. Compare le nombre observé au nombre attendu.
- Implémente le **test de Kupiec** (test statistique formel qui vérifie si le taux d'exceptions observé est statistiquement cohérent avec le taux théorique). Cherche la formule (log-likelihood ratio test) — c'est un bon exercice de compréhension d'un vrai test statistique utilisé en risk management réglementaire (Bâle).
- **Conclusion attendue** : quelle méthode de VaR est la plus fiable sur ton portefeuille ? Justifie avec le résultat du test, pas une impression.

Étape 5 — Composant Machine Learning : détection de régime de volatilité

- Construis une variable cible : `high_vol` = 1 si la volatilité réalisée sur les 5 prochains jours dépasse le 75e percentile historique, sinon 0. (Attention à ne pas te servir du futur pour prédire le futur : les features doivent être connues **avant** le jour de la prédiction.)
- Features possibles (calcule-les toi-même, ne les copie pas d'une lib) : volatilité réalisée sur 5/20 jours glissants, rendement moyen glissant, drawdown courant, VIX si tu veux aller plus loin (`^VIX` sur yfinance).
- Entraîne **une régression logistique ET un random forest** (exactement la comparaison qu'on a faite ensemble) :

- Split train/test **chronologique** (jamais de mélange aléatoire sur une série temporelle — tu testerais le modèle sur du passé par rapport à son entraînement, ce qui fausse tout).
- Calcule le recall par classe, l'AUC, trace la courbe ROC des deux modèles (mêmes outils qu'on a vus ensemble).
- Conclusion attendue : lequel choisis-tu, et pourquoi (relie à ce qu'on a discuté sur l'overfitting et l'interprétabilité en finance) ?

Étape 6 — Mise en forme “production”

C'est ce qui distingue un projet de desk d'un notebook Jupyter isolé :

```
projet-var-desk/
├─ README.md           # objectif, comment lancer, résultats clés
├─ requirements.txt     # dépendances figées (yfinance, pandas, scikit-learn, scipy,
matplotlib, pytest)
├─ config.yaml         # tickers, poids, niveaux de confiance — jamais en dur dans le code
├─ data/
│   └─ processed/
├─ src/
│   ├── data_ingestion.py # téléchargement + nettoyage
│   ├── var_models.py     # les 3 fonctions de VaR
│   ├── backtesting.py    # test de Kupiec + comptage d'exceptions
│   ├── regime_model.py   # feature engineering + logistique + random forest
│   └─ pipeline.py        # orchestre tout, un seul point d'entrée
├─ tests/
│   ├── test_var_models.py
│   └─ test_data_ingestion.py
└─ notebooks/
    └─ exploration.ipynb  # 1'EDA seulement, pas la logique finale
```

- Chaque fonction de `src/` doit avoir une docstring et des tests unitaires simples dans `tests/` (au minimum : la VaR historique sur des données connues donne-t-elle le bon quantile ? La fonction de Kupiec renvoie-t-elle le bon résultat sur un cas simple que tu calcules à la main ?).
- `pipeline.py` doit pouvoir tourner en une seule commande (`python -m src.pipeline`) et produire un rapport (même un simple `report.md` ou des graphiques sauvegardés dans `outputs/`).
- Bonus si tu as le temps : un petit dashboard Streamlit qui affiche la VaR du jour, la matrice de corrélation, et le régime de volatilité détecté.

Programme du week-end (réaliste, pas optimiste)

Samedi matin — Étapes 1 et 2 (ingestion, nettoyage, EDA). Ne sous-estime pas le temps que prend le nettoyage de données réelles.

Samedi après-midi — Étape 3 (les 3 VaR). Vérifie chaque résultat à la main sur un petit échantillon avant de faire confiance à ton code.

Dimanche matin — Étape 4 (backtesting + test de Kupiec). C'est la partie la plus subtile intellectuellement, prends ton temps.

Dimanche après-midi — Étape 5 (régime ML) + début de la mise en forme production (étape 6). Si le temps manque, la structure de dossiers et le `README.md` clair comptent plus qu'un Streamlit complet.

Ce qu'on vérifiera ensemble

Prépare ces éléments précis pour notre session de review : 1. Le tableau comparatif des 3 VaR (95% et 99%). 2. Le résultat du test de Kupiec pour chaque méthode, avec ta conclusion. 3. Les courbes ROC de la régression logistique et du random forest, côte à côte, avec ton choix final justifié. 4. Ta réponse

chiffrée à “les rendements ont-ils des queues épaisses ?” (skewness/kurtosis). 5. La structure de ton dépôt (même incomplète), pour qu’on discute de ce qui manquerait avant une vraie mise en production (logging, scheduling, monitoring des dérives de données...).

Pourquoi ce projet est un bon signal pour un recruteur quant

Il touche exactement aux trois piliers qu’un desk risk regarde : mesure du risque (VaR), validation statistique rigoureuse (backtesting/Kupiec), et anticipation via le ML — avec un vrai souci de mise en production (tests, config externalisée, pas de notebook isolé). C’est le genre de repo GitHub qu’on peut montrer et défendre ligne par ligne en entretien.